

Genetic Algorithm Explained

Universal Genetic Algorithm (GA) Optimization

Stochastic Search Strategy for Analog IC Design

1. What is a Genetic Algorithm?

A Genetic Algorithm (GA) is a search heuristic inspired by Charles Darwin's theory of natural evolution. Instead of relying on gradient descent (which can get trapped in local minima or crash on discontinuous functions), a GA maintains a "population" of multiple possible solutions.

In the context of analog circuit design:

- **Chromosome:** A specific set of circuit parameters (e.g., a vector x containing W, L, R, C values).
- **Population:** A matrix of multiple different circuit configurations.
- **Fitness:** How well a specific circuit meets the target specifications (determined by the Loss function).

The algorithm continuously breeds the best-performing circuits to create a new generation, introducing slight random mutations to explore new design spaces, eventually converging on a global optimum.

2. The Normalized Loss Function (`_compute_loss`)

To determine the "fitness" of a circuit, the optimizer must mathematically penalize any deviation from the target specifications (y_{spec}). Because different specs have vastly different magnitudes (e.g., Phase Margin is ~ 60 , while Power is $\sim 10^{-6}$), the loss must be **normalized**.

For any given simulation output (y_{out}), the scalar penalty L is calculated dynamically based on the constraint type:

1. **Maximize:** Penalizes the circuit only if the achieved value is *less* than the target.

$$\text{If } y_{out} < y_{tgt} : \quad L+ = \frac{|y_{tgt} - y_{out}|}{|y_{tgt}| + 10^{-9}}$$

2. **Minimize:** Penalizes the circuit only if the achieved value is *greater* than the target.

$$\text{If } y_{out} > y_{tgt} : \quad L+ = \frac{|y_{out} - y_{tgt}|}{|y_{tgt}| + 10^{-9}}$$

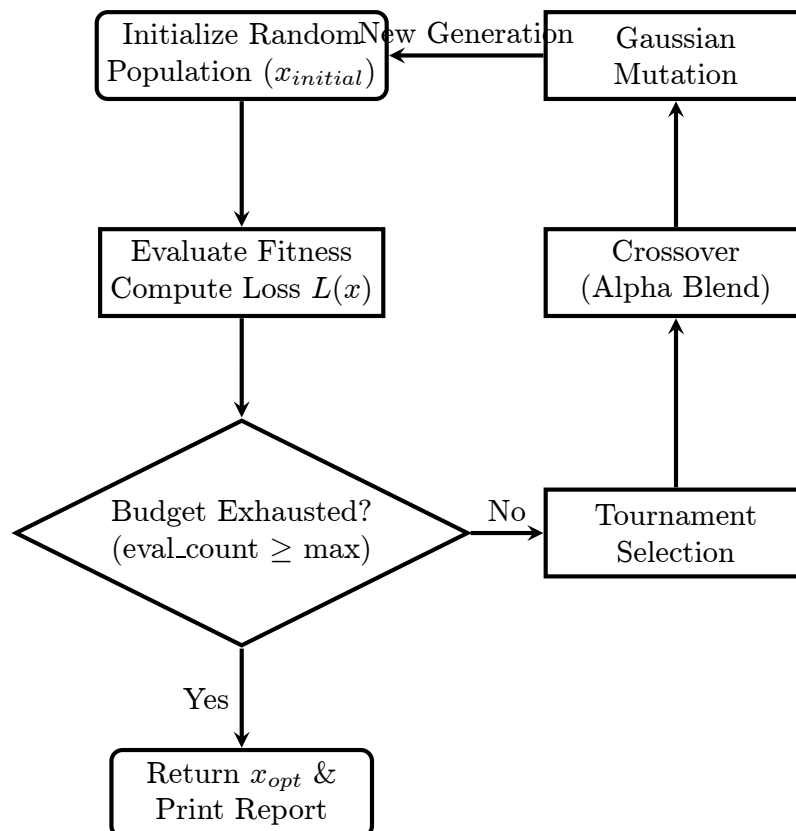
3. **Equate:** Penalizes *any* deviation above or below an exact target matching constraint.

$$L+ = \frac{|y_{out} - y_{tgt}|}{|y_{tgt}| + 10^{-9}}$$

(Note: The 10^{-9} term in the denominator prevents fatal divide-by-zero errors if a target specification is exactly zero).

3. Genetic Algorithm Architecture (Flowchart)

The core loop of the optimizer follows a strict evolutionary pipeline under a defined simulation budget (`max_evals`).



4. Code Implementation Breakdown

The `UniversalGeneticOptimizer` class translates the evolutionary pipeline into a highly robust Python script. Here is how the specific mechanics function:

Elitism & Tracking

During the `_evaluate_individual` phase, the algorithm saves a deep copy of the parameter vector (`self.x_opt`) the moment it encounters a new historical low in the loss function. When building a new generation in `.optimize()`, the absolute best individual from the previous generation is automatically carried over (`new_population = [population[best_idx]]`). This guarantees the optimizer will never accidentally "forget" a good circuit design.

Tournament Selection (`_tournament_selection`)

Instead of ranking the entire population, the algorithm grabs a random subset (default $k = 3$) and selects the one with the lowest loss. This mimics natural competition while maintaining genetic diversity, preventing the population from converging too rapidly on a sub-optimal local minimum.

Continuous Crossover (`_crossover`)

Because circuit parameters are continuous variables (not binary code), the algorithm uses an Alpha-Blend crossover. It takes two parent vectors and creates two children by taking a linear weighted average of their parameters (e.g., $C_1 = 0.5 \cdot P_1 + 0.5 \cdot P_2$).

Gaussian Mutation (`_mutate`)

To introduce new genetic material, each parameter in a child vector has a probability (`mutation_rate = 20%`) of mutating. If triggered, a random Gaussian noise value is added to the parameter. The magnitude of this noise is scaled to 10% of the parameter's total allowable boundaries (`mutation_scale * range_spread`), ensuring mutations are impactful but strictly bounded within physically realizable dimensions.

Reporting (`print_report`)

Upon exhausting the simulation budget, the algorithm outputs a cleanly formatted table comparing the achieved specifications against the requested targets. It evaluates the boolean success of each constraint, allowing an explicit 5% error margin for `equate_keys` to account for simulator precision limits.